

Restaurant Management System – Frontend Documentation

Problem Statement

A mid-sized restaurant needs to digitize its operations to improve efficiency and customer experience. The current manual processes for order-taking, table booking, and kitchen workflow management, and error-prone. The restaurant requires a comprehensive digital solution that enables real-time management of:

- **Table Bookings:** Allow customers to book tables online and staff to view daily reservations
- **Order Management:** Enable waiters to take orders digitally and track their status in real-time
- **Kitchen Workflow:** Provide chefs with a streamlined view of orders and their preparation status
- **User Management:** Allow managers to create and manage staff accounts (waiters, chefs)
- **Menu Display:** Present the restaurant menu to customers in an attractive, easy-to-browse format
- **Role-Based Access:** Ensure each staff member sees only the features relevant to their role

Solution: Build a comprehensive Vue.js frontend application that provides role-based dashboards for customers, waiters, managers, and chefs, enabling seamless restaurant operations with real-time data synchronization.

Project Overview

The **Restaurant Management System** is a modern, single-page application (SPA) built with Vue.js 3 that digitizes core restaurant operations. The system features four distinct role-based interfaces, each tailored to specific user needs:

- **Customer Dashboard:** Browse menu, advance table bookings, view and manage bookings
- **Waiter Dashboard:** Create orders, manage table bookings, update order status
- **Chef Dashboard:** View pending orders, update preparation status, monitor kitchen workflow
- **Manager Dashboard:** Manage staff users (create, view, filter by role)

The application emphasizes real-time operations, intuitive user experience, and comprehensive validation to ensure data integrity across all operations.

Key Features

Authentication & Authorization

- Customer registration with email, phone, and password
- Secure login system with role-based authentication
- Automatic redirection to role-specific dashboards
- Protected routes with navigation guards
- Persistent login state using Vuex store and localStorage

Customer Features

- **Browse Menu:** View all available menu items with images, descriptions, and prices
- **Table Booking:** Book tables for future dates and times
- **Booking Management:** View upcoming bookings and cancel if needed

Waiter Features

- **Order Creation:** Create orders by selecting table and menu items
- **Order Tracking:** View all active orders and their status
- **Order Status Updates:** Update orders from PENDING → PREPARING → READY → SERVED
- **Today's Bookings:** View all table reservations for the current day

Chef Features

- **Kitchen Dashboard:** View all pending and preparing orders
- **Order Prioritization:** Clear visibility of order queue
- **Status Updates:** Mark orders as PREPARING or PREPARED
- **Order Details:** View complete order information including items and table number
- **Visual Indicators:** Color-coded status badges for quick identification

Manager Features

- **User Management:** Create new staff accounts (waiters, chefs)
 - **User Display:** View all users in the system (excluding managers)
 - **Role Filtering:** Filter users by role (ALL, WAITER, CHEF, CUSTOMER)
 - **User Details:** View user information including email and ID
 - **Validation:** Comprehensive form validation for user creation
-

Technology Stack

Frontend Framework & Core

- **Vue.js 3.5.25:** Progressive JavaScript framework with Composition API and `<script setup>` syntax
- **Vue Router 5.0.3:** Official router for SPA navigation and route guards
- **Vuex 4.1.0:** Centralized state management for authentication

UI & Styling

- **Tailwind CSS 4.2.1:** Utility-first CSS framework for rapid UI development
- **@tailwindcss/vite 4.2.1:** Vite plugin for Tailwind CSS integration
- **Lucide Vue Next 0.575.0:** Modern icon set with 1000+ icons
- **Vue Sonner 2.0.9:** Toast notification library for user feedback

HTTP & API

- **Axios 1.13.5:** Promise-based HTTP client for API requests
- **JSON Server 1.0.0-beta.9:** Mock REST API server for development
- **Axios Mock Adapter 2.1.0:** Testing utility for mocking API responses

Build Tools & Development

- **Vite 7.3.1:** Next-generation frontend build tool with HMR
- **@vitejs/plugin-vue 6.0.2:** Official Vite plugin for Vue 3 SFC support

Testing

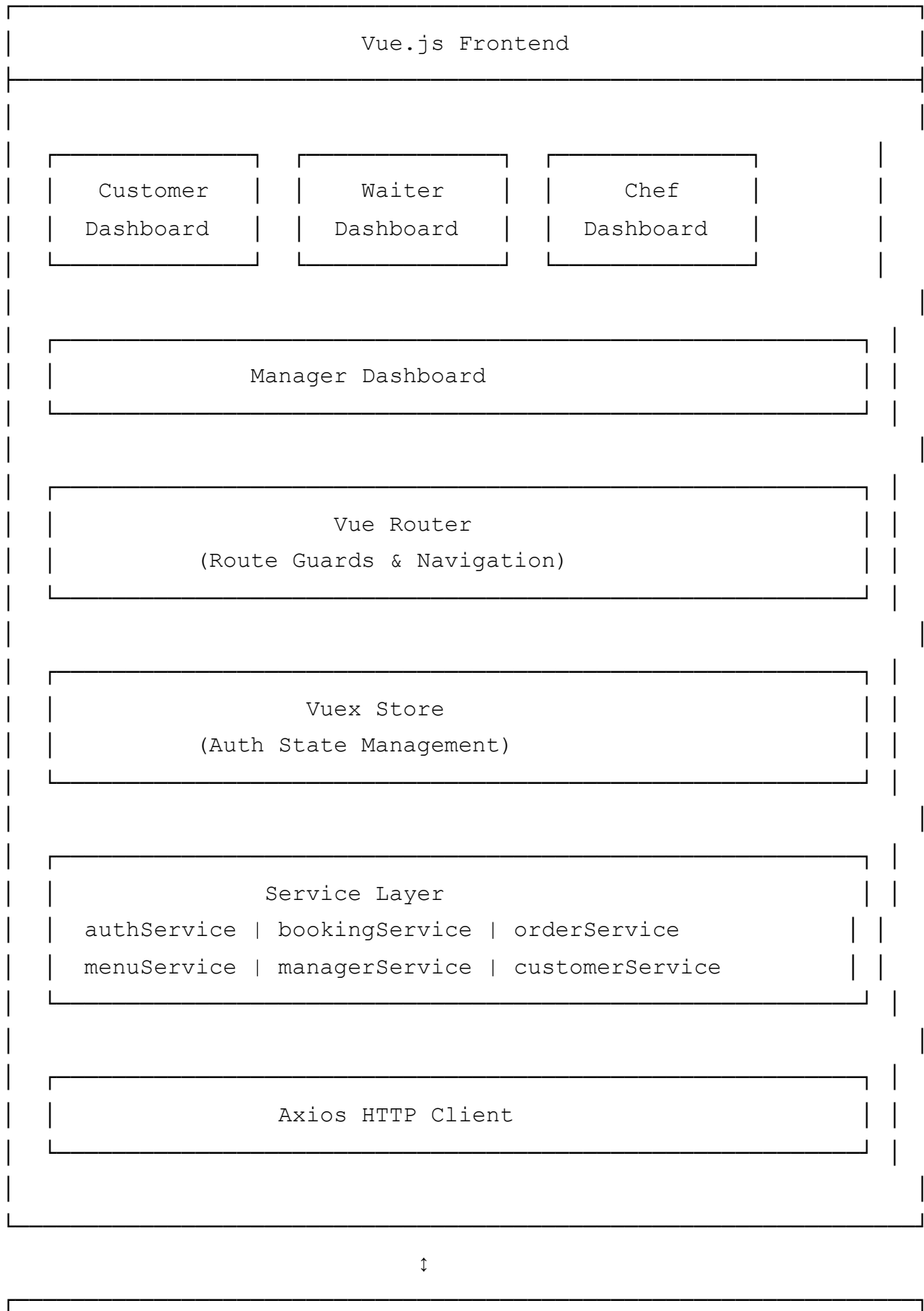
- **Vitest 4.0.18:** Blazing fast unit test framework powered by Vite
- **@vue/test-utils 2.4.6:** Official testing utility library for Vue components
- **@vitest/coverage-v8 4.0.18:** Code coverage tool using V8 JavaScript engine
- **JSDOM 28.1.0:** JavaScript implementation of web standards for testing

JavaScript Environment

- **ES Modules:** Modern module system with `type: "module"` in package.json
 - **Node.js:** Runtime environment for development and build processes
-

System Architecture

Application Architecture



				JSON Server (Backend)		
				db.json		
		users	menuItems	orders	bookings	

Data Flow

- User Action:** User interacts with UI component (form submit, button click)
- State Management:** Component calls Vuex action if auth-related, or service method directly
- HTTP Request:** Service layer makes Axios request to backend API
- Backend Processing:** JSON Server processes request and updates db.json
- Response Handling:** Service returns data to component
- UI Update:** Component updates reactive state, Vue re-renders affected DOM
- User Feedback:** Toast notification displays success/error message

Project Structure

Restaurant-Management-System-Frontend/	
— artifacts/	# Project documentation artifacts
— api_design.pdf	# API endpoint specifications
— Buisness rules.pdf	# Business logic and rules
— wireframes.pdf	# UI/UX wireframes
— src/	
— main.js	# Application entry point
— App.vue	# Root component
— style.css	# Global styles
— assets/	# Images, fonts, static resources
— components/	# Reusable Vue components
— booking/	
— BookTableModal.vue	# Modal for booking tables
— CancelBookingModal.vue	# Modal for canceling bookings
— CustomerBookingsList.vue	# Display customer bookings
— TodayBookingsList.vue	# Display today's bookings
— common/	
— Navbar.vue	# Navigation bar component

```

├─ menu/
│   └─ MenuItem.vue # Menu item card component
├─ order/
│   └─ OrderDetailsModal.vue # Modal for order details
│   └─ WaiterOrdersList.vue # Display waiter's orders
├─ views/ # Page-level components
│   └─ Home.vue # Landing page
│   └─ NotFound.vue # 404 error page
│   └─ auth/ # Authentication pages
│       └─ Login.vue
│       └─ Register.vue
│   └─ dashboards/ # Role-based dashboard pages
│       └─ customer/
│           └─ CustomerDashboard.vue # Customer home
│           └─ CustomerMenu.vue # Menu browsing
│       └─ waiter/
│           └─ WaiterDashboard.vue # Waiter home
│           └─ CreateOrder.vue # Order creation
│       └─ chef/
│           └─ ChefDashboard.vue # Kitchen dashboard
│       └─ manager/
│           └─ ManagerDashboard.vue # Manager home
│           └─ DisplayUser.vue # User list
│           └─ CreateUser.vue # User creation form
├─ router/ # Routing configuration
│   └─ index.js # Route definitions & guards
├─ store/ # Vuex state management
│   └─ index.js # Store configuration
│   └─ modules/
│       └─ auth.js # Auth state, actions, mutations

```

```

| └─ services/                                # API service layer
|   | └─ api.js                               # Axios instance configuration
|   | └─ authService.js                       # Authentication API calls
|   | └─ bookingService.js                   # Booking CRUD operations
|   | └─ orderService.js                     # Order management
|   | └─ menuService.js                      # Menu item fetching
|   | └─ managerService.js                   # User management
|   | └─ customerService.js                  # Customer operations
|   |
| └─ utils/                                   # Utility functions and constants
|   | └─ constants.js                         # App-wide constants
|   | └─ timeUtil.js                         # Date/time formatting
|   | └─ validator.js                        # Input validation functions
|   |
| └─ tests/                                  # Test suites (96 test cases)
|   | └─ auth/                               # Authentication tests
|   |   | └─ authService.spec.js
|   |   | └─ authStore.spec.js
|   |   | └─ login.spec.js
|   |   | └─ register.spec.js
|   |   |
|   | └─ booking/                            # Booking feature tests
|   |   | └─ bookingService.spec.js
|   |   | └─ bookTableModal.spec.js
|   |   | └─ cancelBookingModal.spec.js
|   |   | └─ customerBookingsList.spec.js
|   |   | └─ todayBookingsList.spec.js
|   |   |
|   | └─ dashboards/                         # Dashboard tests
|   |   | └─ chefDashboard.spec.js
|   |   | └─ customerDashboard.spec.js
|   |   | └─ managerDashboard.spec.js
|   |   | └─ waiterDashboard.spec.js
|   |   |
|   | └─ menu/                              # Menu feature tests
|   |   | └─ customerMenu.spec.js
|   |   | └─ menuItem.spec.js
|   |   | └─ menuService.spec.js
|   |   |
|   | └─ order/                             # Order management tests
|   |   | └─ createOrder.spec.js

```

```
|      |   ├── orderService.spec.js
|      |   └── waiterOrdersList.spec.js
|      |
|      ├── user/                                # User management tests
|      |   ├── createUser.spec.js
|      |   ├── displayUser.spec.js
|      |   ├── filterUser.spec.js
|      |   ├── managerService.spec.js
|      |   └── userService.spec.js
|      |
|      └── utils/                                # Utility function tests
|          └── timeUtil.spec.js
|
└── db.json                                     # Mock database (JSON Server)
└── package.json                               # Dependencies and scripts
└── vite.config.js                             # Vite build configuration
└── vitest.config.js                           # Vitest test configuration
└── index.html                                 # HTML entry point
└── README.md                                  # Project readme
```

Database Schema

The application uses **JSON Server** with `db.json` as the mock database. Below is the detailed schema:

Users Collection

```
{
  "users": [
    {
      "id": "string", // Unique identifier (e.g., "W001", "C001")
      "email": "string", // User email (validated)
      "phone": "string", // 10-digit phone number
      "password": "string", // Plain text (demo only)
      "role": "string" // CUSTOMER | WAITER | CHEF | MANAGER
    }
  ]
}
```

Predefined Users for Testing:

ID	Email	Password	Role	Purpose
W001	waiter1@gmail.com	w123	WAITER	Test waiter operations
C001	customer1@gmail.com	c123	CUSTOMER	Test customer features
CH001	chef1@gmail.com	ch123	CHEF	Test chef dashboard
M001	manager@gmail.com	m123	MANAGER	Test manager operations

Menu Items Collection

```
{
  "menuItems": [
    {
      "id": "string",           // Unique identifier (e.g., "1", "2")
      "name": "string",        // Item name
      "description": "string",  // Item description
      "price": number,         // Price in rupees
      "image": "string"        // Image URL
    }
  ]
}
```

Sample Menu Items (15 items total):

- Biryani (₹280)
- Butter Chicken (₹320)
- Paneer Tikka (₹260)
- Masala Dosa (₹150)
- Samosa (₹60)
- Chole Bhature (₹200)
- Pani Puri (₹80)
- Daal Tadka (₹160)
- Rajma Chawal (₹170)
- Aloo Paratha (₹120)
- Pav Bhaji (₹180)
- Idli with Sambar (₹140)
- Gulab Jamun (₹90)
- Jalebi (₹70)
- Paniyaram (₹130)

Orders Collection

```

{
  "orders": [
    {
      "id": "string",          // Auto-generated unique identifier
      "tableId": "string",     // Table number (e.g., "T1", "T5")
      "waiterId": "string",    // ID of waiter who created order
      "items": [               // Array of ordered items
        {
          "itemId": "string",  // Menu item ID
          "quantity": number,   // Quantity ordered
        }
      ],
      "status": "string",      // PENDING | PREPARING | PREPARED | SERVED
      "time": "string"         // ISO datetime string
    }
  ]
}

```

Order Status Flow:

1. **PENDING:** Order created by waiter, awaiting chef action
2. **PREPARING:** Chef is preparing the order
3. **PREPARED:** Food is ready for serving
4. **SERVED:** Order delivered to customer (archived for waiters)

Bookings Collection

```

{
  "bookings": [
    {
      "id": "string",          // Auto-generated unique identifier
      "customerId": "string",  // ID of customer who booked
      "bookingTime": "string", // ISO datetime string
      "numberOfPeople": number, // Number of guests
      "status": "string"       // CONFIRMED | CANCELLED
    }
  ]
}

```

Booking Business Rules:

- Customers can only book future dates/times

- Customers can cancel bookings anytime
 - Waiters can view all bookings for the current day
 - Past bookings are not displayed to customers
-

Role-Based Access & Features

Public Routes (No Authentication Required)

- `/` - Home page
- `/login` - Login page (redirects if authenticated)
- `/register` - Registration page (redirects if authenticated)

Customer Routes (`/customer/*`)

Authentication Required: Yes

Role Required: CUSTOMER

Available Features:

1. **Customer Dashboard** (`/customer`)
 - View all upcoming bookings
 - Book new table with date/time/guest count
 - Edit existing booking (change date/time)
 - Cancel booking with confirmation modal
 - Quick navigation to menu
2. **Customer Menu** (`/customer/menu`)
 - Browse all available menu items
 - View item images, descriptions, and prices

Waiter Routes (`/waiter/*`)

Authentication Required: Yes

Role Required: WAITER

Available Features:

1. **Waiter Dashboard** (`/waiter`)

- View all active orders (PENDING, PREPARING, PREPARED)
- Update order status via dropdown
- View order details in modal
- View today's bookings
- Quick navigation to create order

2. Create Order (/waiter/create-order)

- Select table number
- Browse menu items
- Add/remove items with quantity controls
- View live order summary (total items, total amount)
- Submit order
- Reset functionality

Chef Routes (/chef/*)

Authentication Required: Yes

Role Required: CHEF

Available Features:

1. Chef Dashboard (/chef)

- View all PENDING and PREPARING orders
- Order cards with table number and items list
- Update order status (PENDING → PREPARING → PREPARED)
- Visual status indicators
- Order stats
- Empty state when no orders

Manager Routes (/manager/*)

Authentication Required: Yes

Role Required: MANAGER

Available Features:

1. Manager Dashboard (/manager)

- Navigation sidebar with user management options
- View all users
- Create new user
- Filter users by role

2. Display Users (`/manager` or `/manager/users`)

- Table view of all users (excluding managers)
- Filter by role: ALL, WAITER, CHEF, CUSTOMER
- Display user ID, email, role
- Responsive design

3. Create User (`/manager/create-user`)

- Form with email, phone, password, role selection
 - Validation:
 - All fields required
 - Valid email format
 - Valid 10-digit phone number
 - Check for duplicate email
 - Auto-generate user ID based on role
 - Success feedback and navigation
-

Installation & Setup

Prerequisites

Ensure you have the following installed on your system:

- **Node.js:** v18.0.0 or higher ([Download](#))
- **npm:** v9.0.0 or higher (comes with Node.js)
- **Git:** For version control ([Download](#))

Step 1: Extract the Project

If you received the project as a zip file:

```
# Extract the zip file
unzip Restaurant-Management-System-Frontend.zip

# Navigate to the project directory
cd Restaurant-Management-System-Frontend
```

Alternative: If cloning from repository:

```
git clone <repository-url>
cd Restaurant-Management-System-Frontend
```

Step 2: Install Dependencies

```
npm install
```

This will install all dependencies specified in `package.json` :

- Production dependencies (Vue, Vue Router, Vuex, Axios, etc.)
- Development dependencies (Vite, Vitest, testing utilities)

Step 3: Environment Configuration

Create a `.env` file in the root directory (optional):

```
VITE_BACKEND_URL=http://localhost:3000
```

Note: If not specified, the application defaults to `http://localhost:3000`.

Step 4: Verify Installation

Check that all dependencies are installed correctly:

```
npm list --depth=0
```

You should see all packages listed in `package.json` without errors.

Running the Application

Development Mode

Start Backend Server (JSON Server)

In one terminal:

```
npm run json-server
```

Or use the full command:

```
npx json-server db.json --port 3000
```

Output:

Resources

```
http://localhost:3000/users
http://localhost:3000/menuItems
http://localhost:3000/orders
http://localhost:3000/bookings
```

Start Frontend Development Server

In another terminal:

```
npm run dev
```

Output:

```
VITE v7.3.1  ready in 250 ms
```

```
→ Local:    http://localhost:5173/
```

```
→ Network:  use --host to expose
```

Access the Application

Open your browser and navigate to:

```
http://localhost:5173
```

Production Build

Build the application for production:

```
npm run build
```

This creates an optimized production build in the `dist/` directory.

Preview Production Build

Preview the production build locally:

```
npm run preview
```

Testing

The project includes a comprehensive test suite with **96 test cases** covering all major features.

Test Framework & Tools

- **Vitest**: Fast unit test framework
- **@vue/test-utils**: Vue component testing utilities
- **JSDOM**: Browser environment simulation
- **Axios Mock Adapter**: API mocking for tests

Test Structure

src/tests/	
├─ auth/	Login, register, auth service, auth store
├─ booking/	Booking CRUD, modals, lists
├─ dashboards/	All role dashboards
├─ menu/	Menu display, menu items
├─ order/	Order creation, management
├─ user/	User management, filtering
└─ utils/	Utility functions

Running Tests

Run All Tests

```
npm test
```

Run Tests in Watch Mode

```
npm run test --watch
```


Run Tests with Coverage

```
npm run coverage
```

This generates a coverage report in the `coverage/` directory.

Test Coverage Report

After running `npm run coverage`, you'll see a summary in the terminal and can open `coverage/index.html` in your browser for detailed coverage information.

Expected Coverage:

- Statements: >80%
- Branches: >70%
- Functions: >70%
- Lines: >80%

Test Results

Test Execution Summary

```
✓ src/tests/auth/login.spec.js (3 tests) 93ms
✓ src/tests/dashboards/waiterDashboard.spec.js (5 tests) 90ms
stderr | src/tests/order/createOrder.spec.js > CreateOrder - order submission > resets all quantities
[Vue Router warn]: No match found for location with path ""

✓ src/tests/order/createOrder.spec.js (8 tests) 108ms
✓ src/tests/dashboards/customerDashboard.spec.js (4 tests) 107ms
✓ src/tests/dashboards/chefDashboard.spec.js (5 tests) 102ms
✓ src/tests/user/filterUser.spec.js (2 tests) 17ms
✓ src/tests/booking/bookTableModal.spec.js (5 tests) 25ms
✓ src/tests/booking/customerBookingsList.spec.js (4 tests) 31ms
✓ src/tests/order/waiterOrdersList.spec.js (5 tests) 32ms
✓ src/tests/auth/register.spec.js (3 tests) 79ms
✓ src/tests/menu/customerMenu.spec.js (4 tests) 71ms
✓ src/tests/utlis/timeUtil.spec.js (3 tests) 10ms
✓ src/tests/menu/menuItem.spec.js (2 tests) 8ms
✓ src/tests/booking/cancelBookingModal.spec.js (4 tests) 20ms
✓ src/tests/dashboards/managerDashboard.spec.js (3 tests) 10ms
✓ src/tests/user/displayUser.spec.js (3 tests) 16ms
✓ src/tests/booking/todayBookingsList.spec.js (3 tests) 9ms
✓ src/tests/booking/bookingService.spec.js (5 tests) 2ms
✓ src/tests/auth/authService.spec.js (6 tests) 2ms
✓ src/tests/auth/authStore.spec.js (7 tests) 4ms
✓ src/tests/user/managerService.spec.js (1 test) 2ms
✓ src/tests/user/userService.spec.js (2 tests) 4ms
✓ src/tests/menu/menuService.spec.js (2 tests) 4ms
✓ src/tests/order/orderService.spec.js (3 tests) 1ms

Test Files 25 passed (25)
Tests 96 passed (96)
Start at 16:12:26
Duration 3.07s (transform 498ms, setup 0ms, import 2.45s, tests 937ms, environment 11.56s)

PASS Waiting for file changes...
press h to show help, press q to quit
```

Description: Complete test execution showing all 96 test cases passing with execution time.

Code Coverage Report

All files
85.06% Statements 558/656 72.59% Branches 249/343 71.87% Functions 115/160 84.97% Lines 543/639

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
components/booking	81.3%	100/123	77.19%	44/57
components/menu	100%	10/10	100%	0/0
components/order	76.34%	71/93	22.22%	10/45
services	86.95%	60/69	74.19%	23/31
store	100%	1/1	100%	0/0
store/modules	97.22%	35/36	50%	4/8
utlis	100%	10/10	100%	0/0
views/auth	95.34%	41/43	93.47%	43/46
views/dashboards/chef	86.66%	52/60	86.66%	26/30
views/dashboards/customer	74.62%	50/67	57.69%	15/26
views/dashboards/manager	85.96%	49/57	79.54%	35/44
views/dashboards/waiter	90.8%	79/87	87.5%	49/56

Description: Vitest coverage report showing statement, branch, function, and line coverage percentages across all modules.

Test Categories Breakdown

Category	Test Files	Test Cases	Description
Authentication	4	19	Login, register, auth service tests
Bookings	5	21	Table booking and management tests
Dashboards	4	17	All role-based dashboard tests
Menu	3	8	Menu display and item tests
Orders	3	16	Order creation and management tests
Users	5	12	User management and filtering tests
Utils	1	3	Utility function tests
TOTAL	25	96	Complete test coverage

Project Artifacts

The `artifacts/` directory contains essential project documentation created during the planning and design phase:

1. API Design Document

File: [artifacts/api_design.pdf](#)

Contents:

- Complete REST API endpoint specifications
- Request/response formats for all operations
- HTTP methods and status codes
- Query parameters and request bodies
- Error handling specifications
- Authentication and authorization requirements

Endpoints Covered:

- `/users` - User authentication and management
- `/menuItems` - Menu item retrieval
- `/orders` - Order CRUD operations
- `/bookings` - Booking management

2. Business Rules Document

File: [artifacts/Buisness rules.pdf](#)

Contents:

- Restaurant operation workflows
- Role-based access control rules
- Order status transition rules
- Booking validation rules
- User creation and management policies
- Data validation requirements
- Business constraints and limitations

Key Business Rules:

- Customers can only book future dates
 - Orders progress: PENDING → PREPARING → READY → SERVED
 - Managers cannot delete manager accounts
 - Waiters can only see their own orders
 - Chefs see all pending/preparing orders
 - Email uniqueness per role
-

3. Wireframes Document

File: [artifacts/wireframes.pdf](#)

Contents:

- UI/UX wireframes for all pages
- User flow diagrams
- Component layout specifications
- Navigation patterns
- Modal and dialog designs

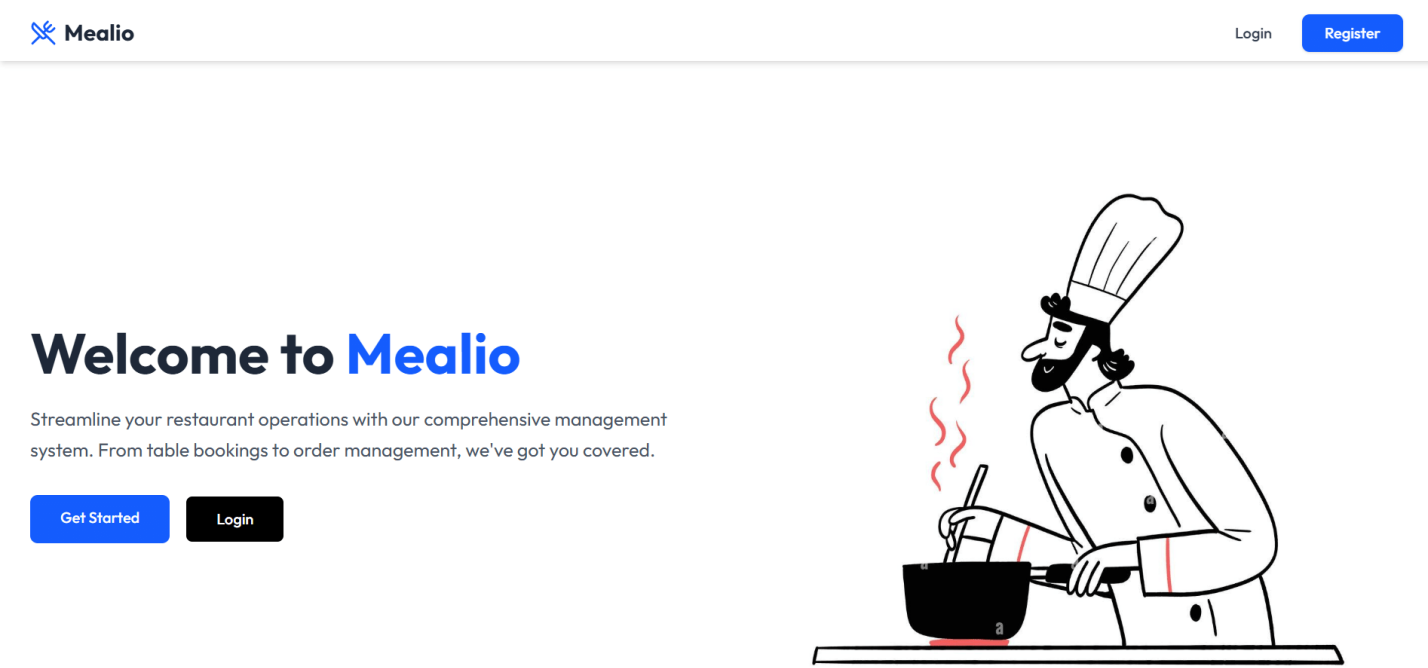
Pages Covered:

- Login and Registration pages
 - Customer Dashboard and Menu
 - Waiter Dashboard and Create Order
 - Chef Dashboard
 - Manager Dashboard and User Management
 - Booking and Order modals
-

UI Screenshots

Authentication Pages

Home Page



Description: Landing page with navigation to login/register. Features welcome message and quick links to access the restaurant management system. Responsive design with clean navigation bar.

Login Page

! Invalid credentials

User Login

Email

nirmith10@gmail.com

Role

Customer



Password

.....

Login

Don't have an account? [Register](#)

Description: User login interface with email, password, and role selection dropdown. Includes form validation, error messaging, and "Register" link. Clean, centered card layout with brand colors.

Register Page

✓ Registration successful! Redirecting...

User Registration

Email

Phone

Password

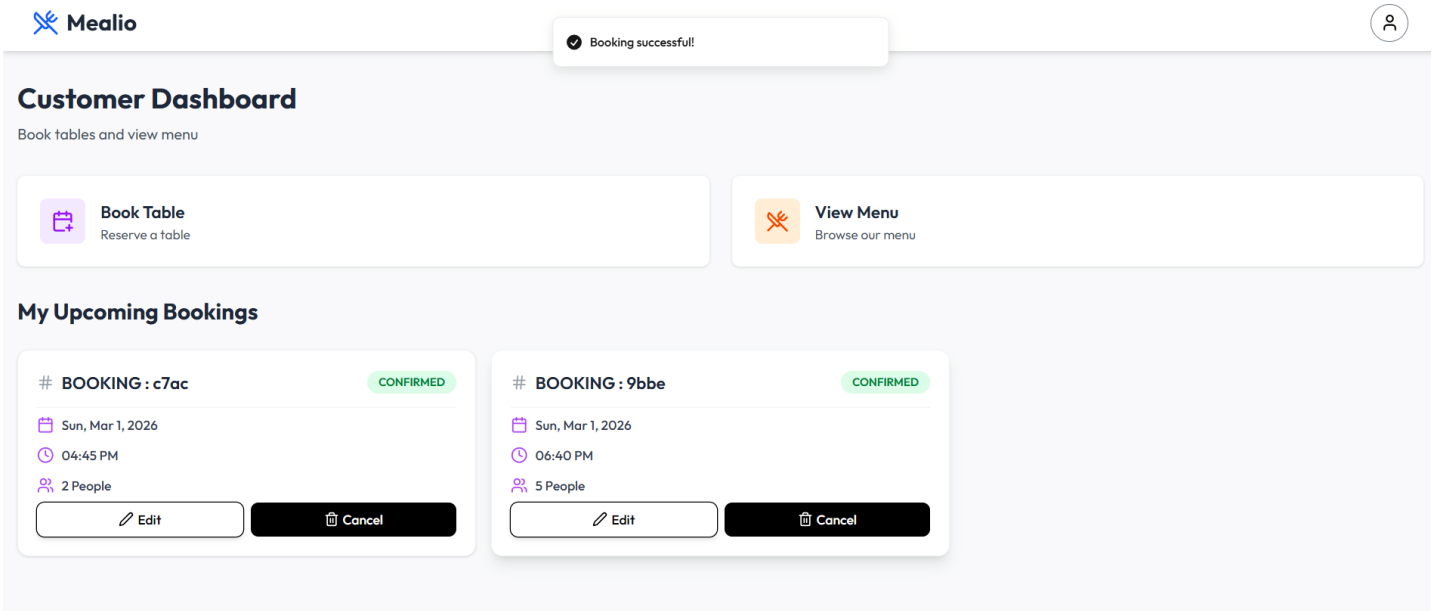
Register

Already have an account? [Login](#)

Description: Customer registration form with email, phone number, and password. Features real-time validation with error messages for invalid inputs. Success toast notification on successful registration.

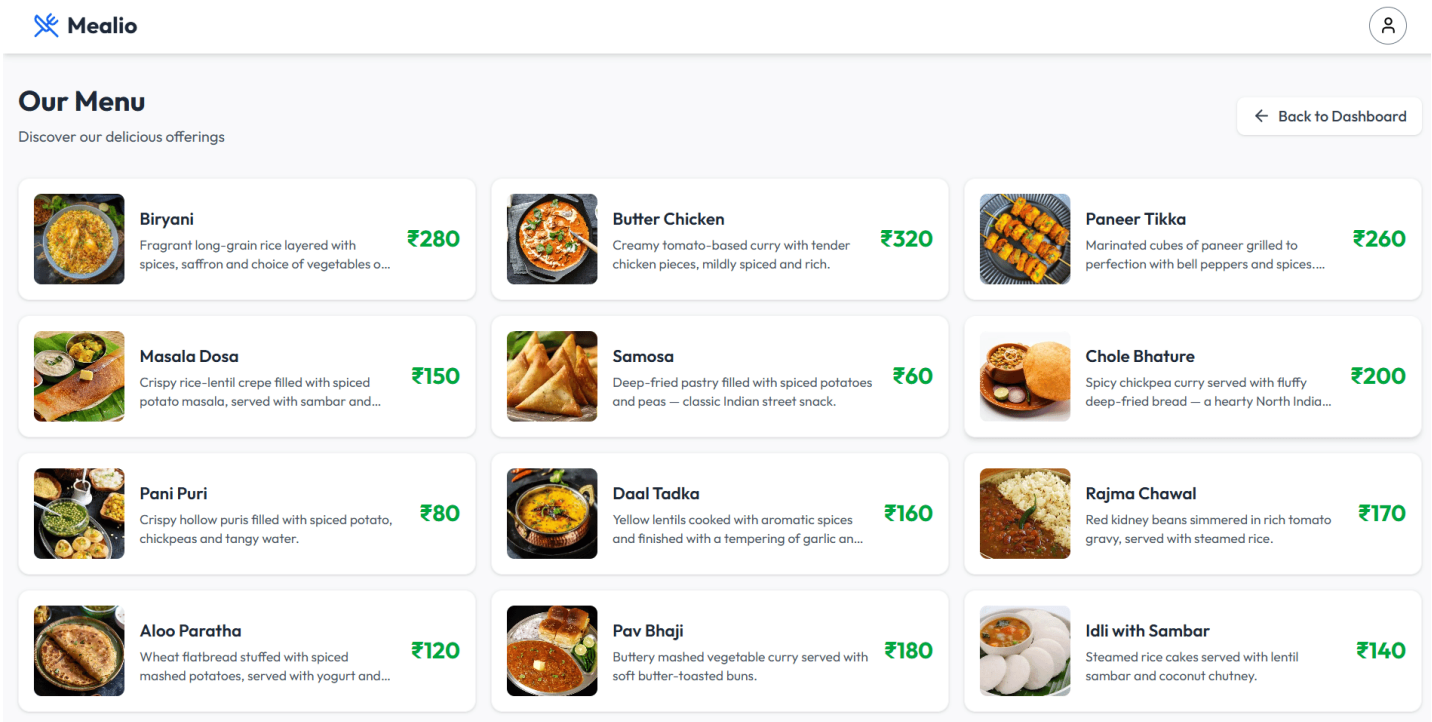
Customer Dashboard

Customer Dashboard - Main View



Description: Customer home page displaying upcoming bookings in card format. Shows booking details (date, time, guest count) with Edit and Cancel buttons. Includes "Book a Table" button and navigation to menu. Empty state shown when no bookings exist.

Customer Menu Page



Description: Grid layout of menu items with images, names, descriptions, and prices. Responsive card design with hover effects. Displays all 15 menu items with proper formatting (₹ symbol).

Book Table Modal

 **Book a Table**

 Number of People

 Booking Date



 Booking Time



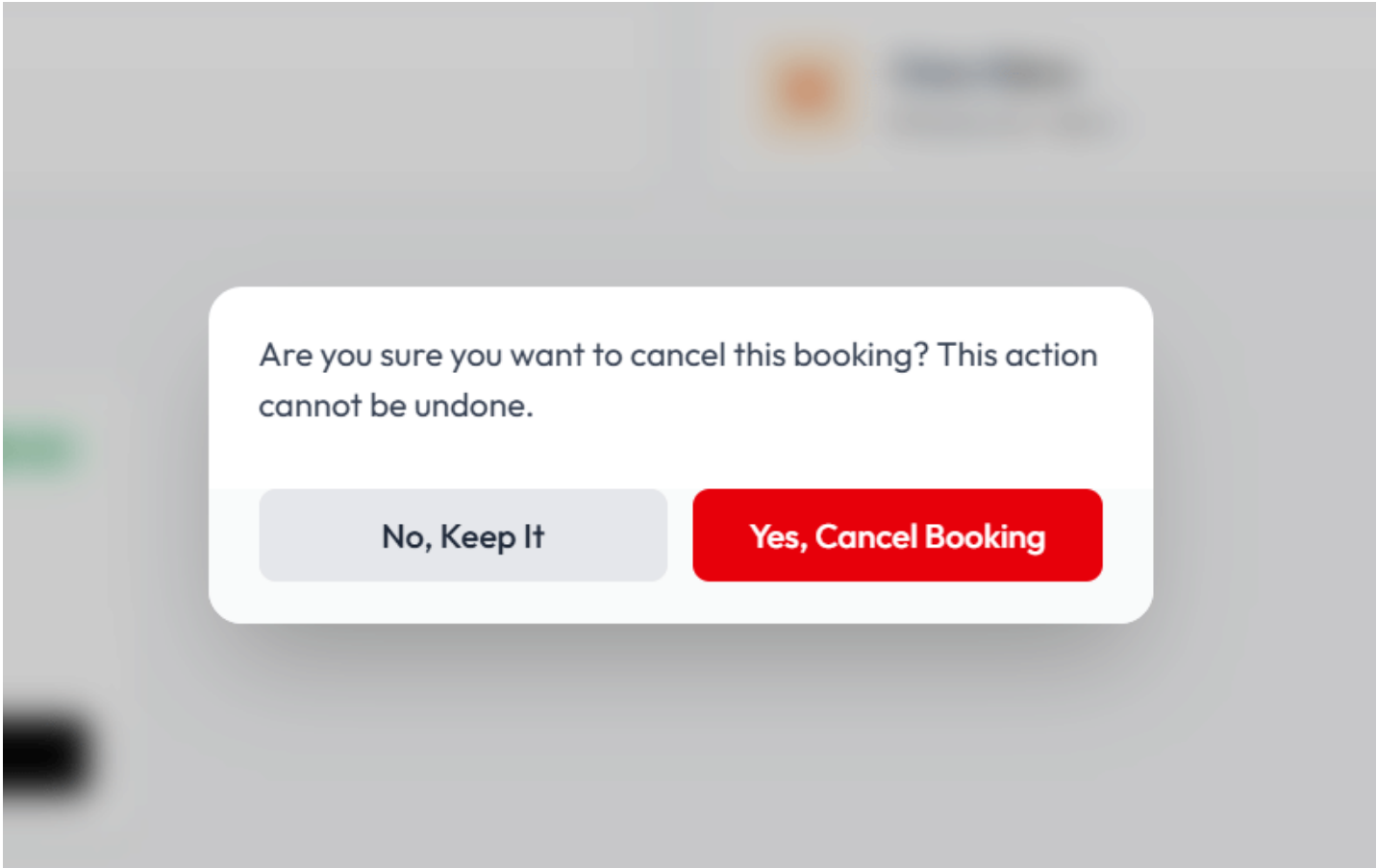
Booking date and time cannot be in the past

Cancel

Book Table

Description: Modal dialog for booking tables with date/time picker and guest count input. Validation prevents past date selection. Shows success message on booking creation and updates the bookings list immediately.

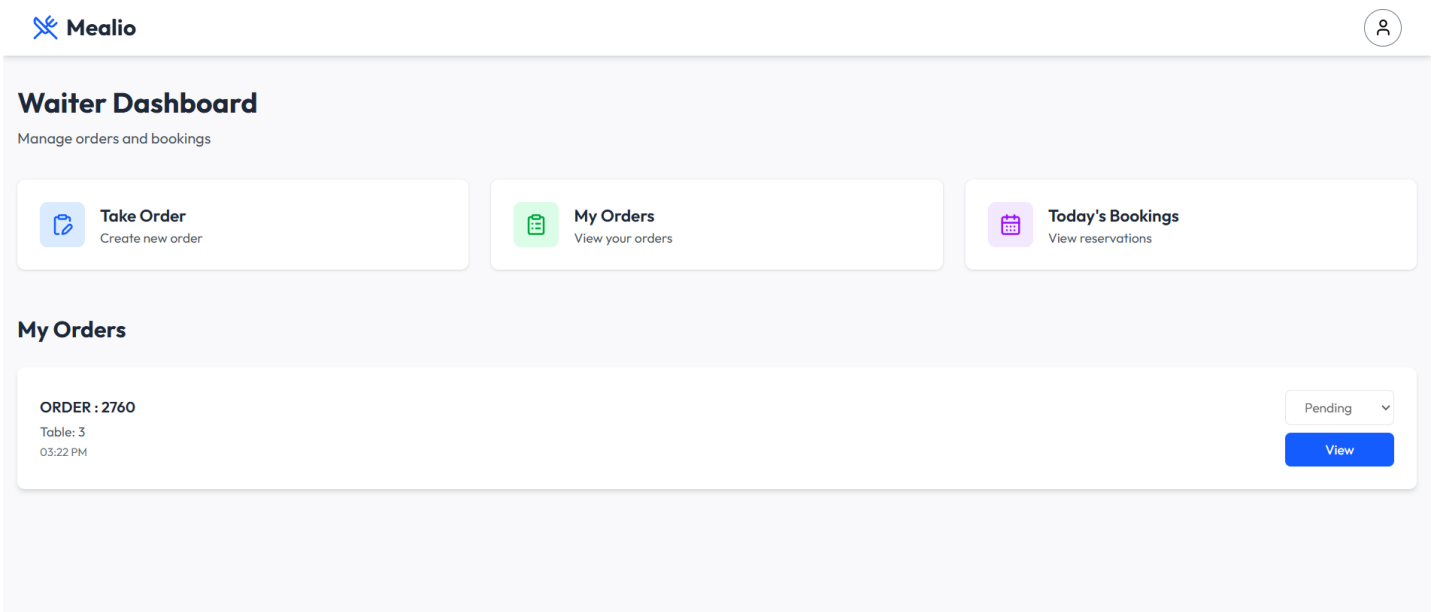
Cancel Booking Modal



Description: Confirmation dialog for booking cancellation with "Yes, Cancel Booking" and "No, Keep It" options. Clear warning message explaining the action is irreversible.

Waiter Dashboard

Waiter Dashboard - Main View



Description: Waiter home showing three main action cards: "View Orders", "Create Order", and "Today's Bookings". Clean navigation with icon indicators. Active orders list displayed with status badges and action buttons.

Create Order Page





Create Order

waiter1 (ID : W001)

Select Table Number:

Table 1

Menu Items



Biryani

Fragrant long-grain rice layered with spices, saffron and choice of vegetables or meat.

₹280

-

1

+



Butter Chicken

Creamy tomato-based curry with tender chicken pieces, mildly spiced and rich.

₹320

-

0

+



Paneer Tikka

Marinated cubes of paneer grilled to perfection with bell peppers and spices. Vegan can be requested.

₹260

-

0

+



Masala Dosa

Crispy rice-lentil crepe filled with spiced potato masala, served with sambar and chutneys.

₹150

-

2

+



Samosa

Deep-fried pastry filled with spiced potatoes and peas — classic Indian street snack.

₹60

-

0

+



Chole Bhature

Spicy chickpea curry served with fluffy deep-fried bread — a hearty North Indian favorite.

₹200

-

0

+



Pani Puri

Total Items: 3

Total Amount: ₹580

Place Order

Description: Order creation interface with table selection dropdown and menu items grid. Each item has +/- quantity controls. Live order summary shows selected total items, and total amount. Submit button at bottom.

Today's Bookings List





Waiter Dashboard

Manage orders and bookings



Take Order

Create new order



My Orders

View your orders



Today's Bookings

View reservations

Today's Bookings

BOOKING : 2618

CONFIRMED

customer1@gmail.com

9876543210

03:24 PM

2 People

BOOKING : acd8

CANCELLED

customer1@gmail.com

9876543210

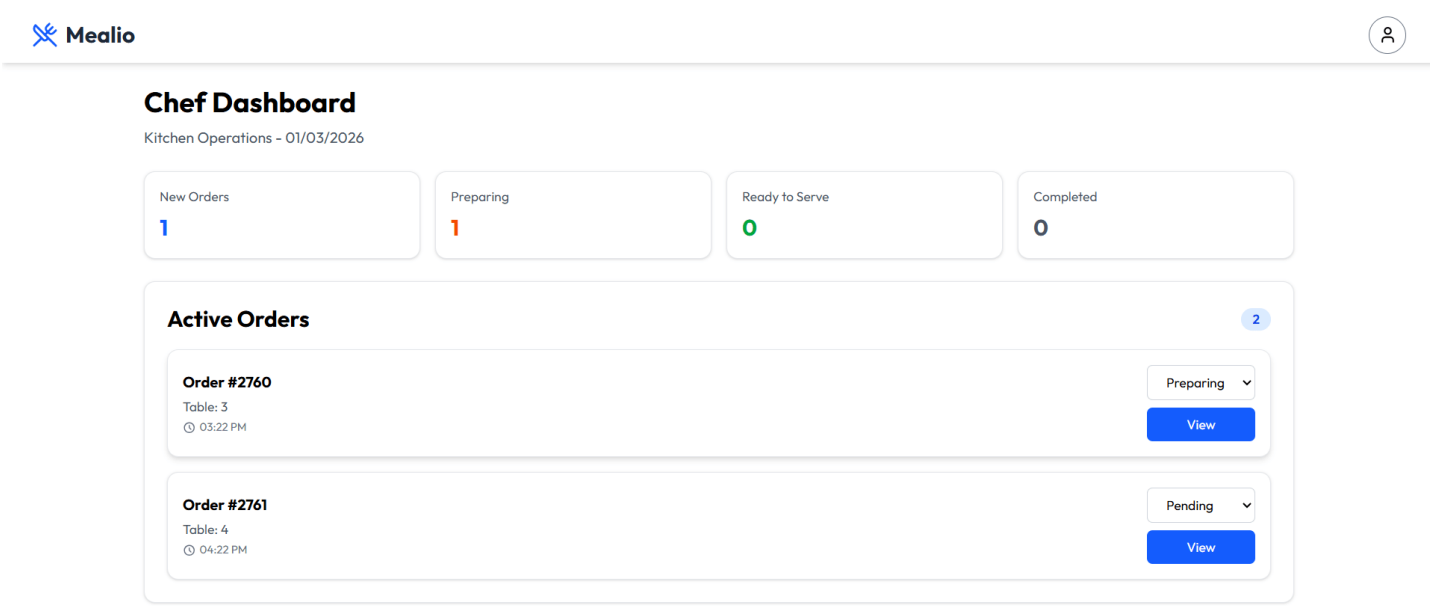
04:32 PM

5 People

Description: List of all bookings for the current day showing customer details, booking time, guest count, and status. Helps waiters prepare for expected guests. Shows formatted time (12-hour format) and date.

Chef Dashboard

Chef Dashboard - Kitchen View



Description: Kitchen-focused dashboard showing all pending and preparing orders. Order cards display table number, order ID, items list, and status update buttons. Color-coded status badges (yellow for PENDING, blue for PREPARING, green for READY). Order counter at top shows pending vs total orders.

Chef Dashboard - Order Details

Description: Manager home with sidebar navigation ("All Users", "Create User"). Main content shows user management table with columns: User ID, Email, Role. Role filter dropdown (ALL, WAITER, CHEF, CUSTOMER) for filtering displayed users.

Create User Page



Manager Panel

Create User

All Users

Please enter a valid phone number

Create User

Add new staff member to the system

Email Address

nirmith10@gmail.com

Password

.....

Phone Number

7005848

Role

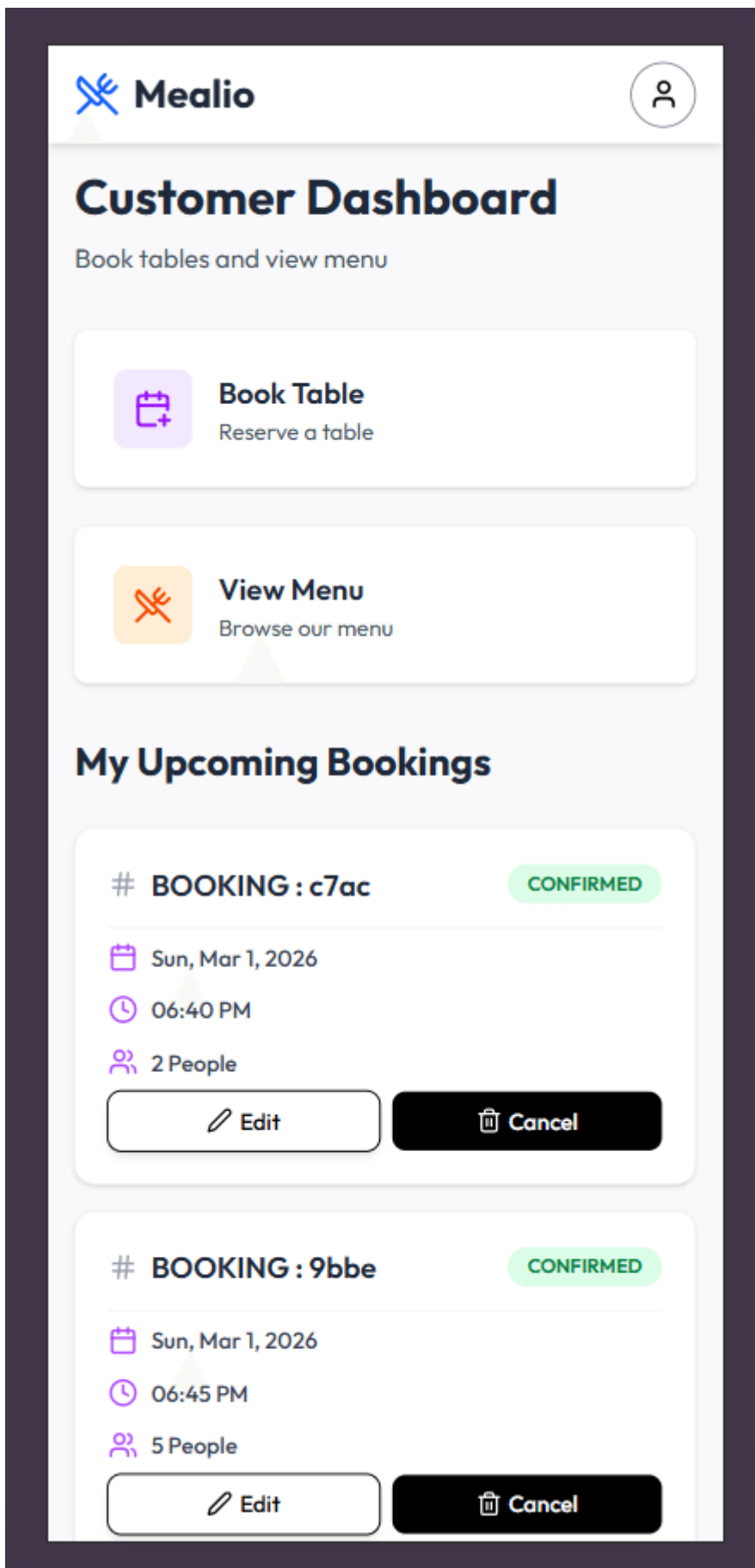
Waiter

Create User

Description: User creation form for managers with fields: Email, Phone Number, Password, and Role dropdown. Comprehensive validation with error messages. Auto-generates user IDs based on role (W00X for waiters, C00X for customers, etc.). Success notification and redirect on user creation.

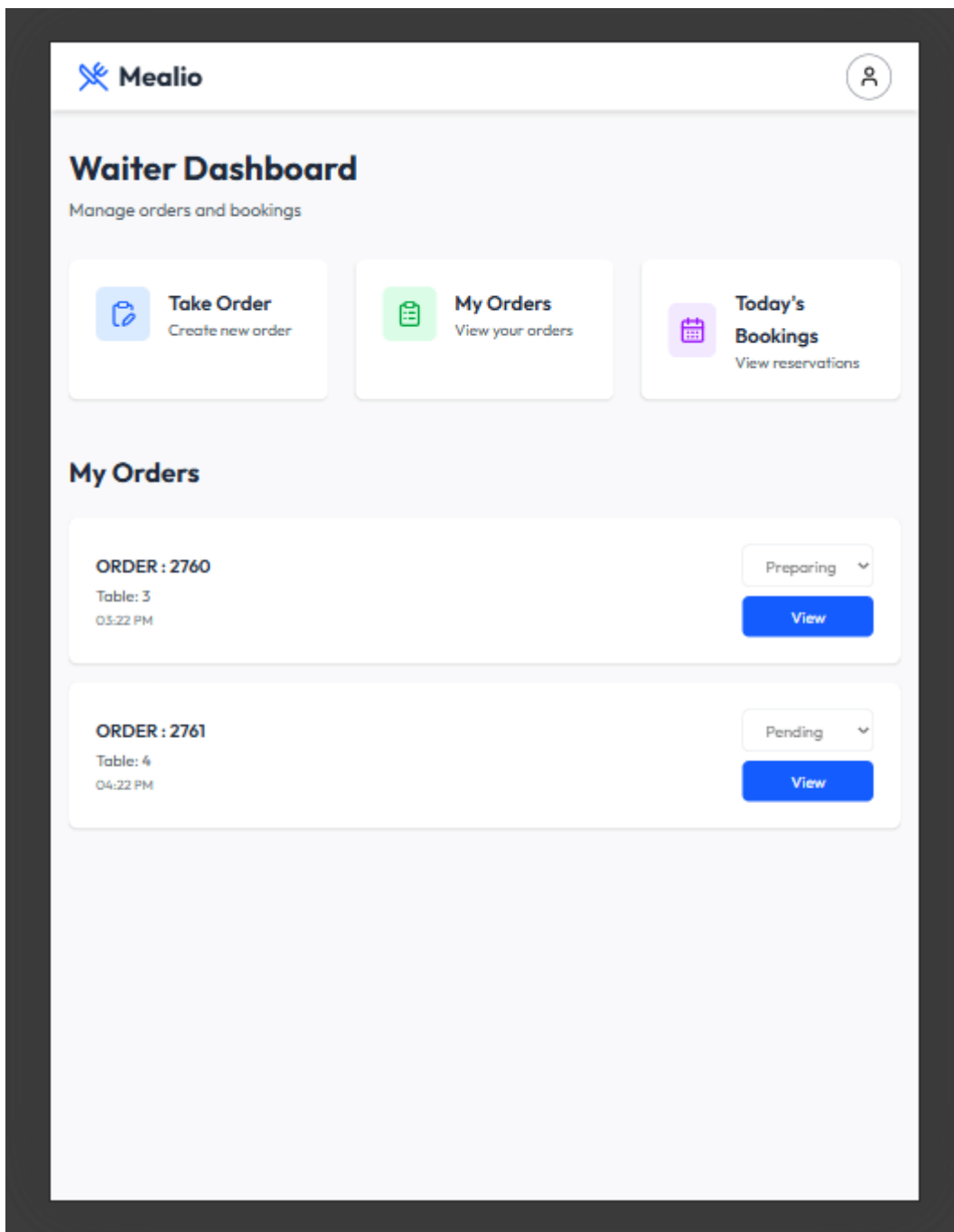
Responsive Design

Mobile View - Customer Dashboard



Description: Responsive design on mobile devices showing customer dashboard. Cards stack vertically, navigation collapses to hamburger menu, and touch-friendly button sizing. Maintains full functionality on smaller screens.

Tablet View - Waiter Dashboard



Description: Medium screen layout on tablet showing waiter dashboard. Optimized column layout for tablet dimensions. Navigation adapts to available screen space while maintaining usability.

API Integration

API Base URL Configuration

The application uses Axios for HTTP requests with a centralized configuration:

File: `src/services/api.js`

```
import axios from "axios";
```



```
const axiosInstance = axios.create({
  baseURL: import.meta.env.VITE_BACKEND_URL || "http://localhost:3000",
  headers: {
    "Content-Type": "application/json",
  },
});

export default axiosInstance;
```

Service Layer Architecture

All API calls are abstracted in service modules for better separation of concerns:

Authentication Service (`authService.js`)

- `login(credentials)` - Authenticate user
- `register(userData)` - Register new user
- Includes validation (email format, phone number, duplicate check)

Booking Service (`bookingService.js`)

- `getTodayBookings()` - Get all bookings for current day
- `getUpcomingBookingsByCustomerId(customerId)` - Get customer's future bookings
- `createBooking(bookingData)` - Create new booking
- `updateBooking(bookingId, bookingData)` - Modify existing booking
- `cancelBooking(bookingId)` - Cancel booking

Order Service (`orderService.js`)

- `getAllOrders()` - Get all orders (for chefs)
- `getOrdersByWaiterId(waiterId)` - Get waiter's orders (excludes SERVED)
- `createOrder(order)` - Create new order
- `updateOrderStatus(id, status)` - Update order status

Menu Service (`menuService.js`)

- `getMenuItems()` - Fetch all menu items

Manager Service (`managerService.js`)

- `getUsers()` - Get all users except managers

Customer Service (`customerService.js`)

- `getUserById()` - Gets user by their unique id
-